Повторення Arduino і функції з параметрами

Урок 1 — 2МАК, SST-INF-01c — Мікропрограми (firmware): що пам'ятати з минулого року, і функції

15/10/2026

Примітка

Цей роздатковий матеріал є також **підсумком** того, що ми робили з firmware минулого року: хто прийшов цього року, знайде тут усе потрібне, щоб наздогнати клас. **29 жовтня** (зранку) заплановано **контрольну з повторення** firmware, приблизно на дві години, з усіх тем минулого року і тем цього уроку.

1 Структура програми Arduino

Програма Arduino має два основні блоки:

- `setup()`: містить **початкові налаштування**, команди, які виконуються **лише один раз**, під час увімкнення;
- `loop()`: містить частину, яка має виконуватися **безперервно**, по колу, поки плата ввімкнена.

2 Входи і виходи: `pinMode`, `digitalWrite`, `digitalRead`

Приклад

Двері супермаркету. У багатьох супермаркетах двері односторонні: через одні заходять, через інші виходять. Arduino теж має "двері" — **виводи** (*piedini*): через одні інформація **входить** (**input**, вхід), через інші **виходить** (**output**, вихід). Якщо це вихід, то Arduino вирішує, що випускати: **увімкнено чи вимкнено**. Якщо це вхід, Arduino не вирішує, що надходить: це **зовнішній світ** каже, є на виводі сигнал чи ні.

Інший спосіб це уявити: щоб говорити, ти використовуєш **рот**, щоб слухати — **вуха**. Не можна говорити вухами чи слухати ротом. Якщо вивід 12 — вхід, ти можеш "почути", що каже зовнішній світ на 12-му, але не можеш сам вирішувати, що туди надійде.

Визначення

`pinMode(вивід, OUTPUT)` або `pinMode(вивід, INPUT)` задає **напрямок** виводу: чи це вихід, чи вхід. Він задає *лише* напрямок, а не те, що через нього проходить.

Примітка

Зазвичай `pinMode` пишуть **на початку**, у `setup()`: якщо вирішили, що вивід — вихід, він залишається виходом. У складніших пристроях є виводи, які якийсь час працюють як input, а

якийсь час як output, і тоді `pinMode` ставлять також у `loop()`, там, де треба змінити напрямок. Можливо, розглянемо це наступного року.

Визначення

На виводі, налаштованому як **output**, можна лише вирішити, подавати напругу чи ні: **`digitalWrite(вивід, HIGH)`** (увімкнено) або **`digitalWrite(вивід, LOW)`** (вимкнено).
На виводі, налаштованому як **input**, "слухають" за допомогою **`digitalRead(вивід)`**, який повідомляє, високий чи низький рівень на виводі. Прочитане значення треба **кудись записати**, зазвичай у змінну:

```
int x = digitalRead(12);
```

Увага

Ці три команди — `pinMode` (напрямок), `digitalWrite` (записати на вихід), `digitalRead` (прочитати вхід) — мають міцно сидіти в пам'яті: це основа всього, що ми робитимемо.

3 Час: `delay` і `delayMicroseconds`

Визначення

`delay(t)` **зупиняє** машину на певний час: вона стоїть `t` **мілісекунд**. `delay(1000)` означає зупинитися на 1000 тисячних секунди, тобто на **одну секунду**. Найменше корисне значення — 1 (одна мілісекунда): `delay` приймає лише **цілі числа**, тож не можна написати `delay(0.5)` для пів мілісекунди.

Примітка

Для часу, коротшого за мілісекунду, є **`delayMicroseconds(t)`**, яка працює так само, але задає час у **мікросекундах** (мільйонних частках секунди): `delayMicroseconds(1000)` зупиняє машину на одну тисячну секунди.
Уточнення до уроку: якщо все ж написати `delay(0.5)`, програма скомпілюється, але значення **відкинеться** до 0, і паузи не буде.

4 Послідовний зв'язок: `Serial`

`Serial` — це набір ("клас") команд, який керує **зв'язком** між Arduino і комп'ютером. Ті, що ми бачили минулого року:

- **`Serial.begin(швидкість)`**: задає **швидкість** обміну даними. Arduino і комп'ютер (або будь-який пристрій, що отримує дані) мають бути налаштовані на **однакову** **швидкість**, інакше вони не зрозуміють одне одного: це не відбувається автоматично, ви маєте налаштувати це самі. Типові значення — **9600** і **115200**. Не обов'язково завжди ставити максимум.
- **`Serial.print(...)`** надсилає повідомлення в консоль Arduino; наступне повідомлення про-

довжується **в тому самому рядку**.

- **`Serial.println(...)`** робить те саме, але після повідомлення **переходить на новий рядок** (*ln = line*).
- **`Serial.available()`** показує, **скільки повідомлень** (символів) прийшло і **чекає**, щоб їх прочитали.
- **`Serial.read()`** **читає** перший символ, що чекає, і **прибирає його з черги**.

Приклад

Віконце і черга. Коли комп'ютер надсилає дані в Arduino, вони потрапляють у пам'ять, яка називається **буфер** (*buffer*). Це як установа з віконцем: хто приходить, поки віконце зайняте, стає **в чергу** і чекає. Надсилання залежить не від Arduino, а від того, хто надсилає дані: Arduino може бути скільки завгодно швидким, але йому потрібен час, щоб обробити кожне значення, тому він мусить відкладати ті, що надходять тим часом. `Serial.available()` каже, **скільки людей у черзі**; `Serial.read()` **пропускає до віконця** першого з черги.

Приклад

Типова програма: Arduino безперервно пише "ciao" в консоль; якщо з комп'ютера приходить літера `x`, він відповідає "premuto" ("натиснуто").

```
void setup() {
  Serial.begin(9600);
}

void loop() {
  Serial.println("ciao");
  if (Serial.available() > 0) {   // чи є хтось у черзі?
    char dato = Serial.read();    // читаю і прибираю з черги
    if (dato == 'x') {
      Serial.println("premuto");
    }
  }
  delay(500);   // без паузи повідомлення бігли б надто швидко
}
```

Залежно від значення `dato` програма вирішує, що робити: це залежить від того, що ми хочемо зробити.

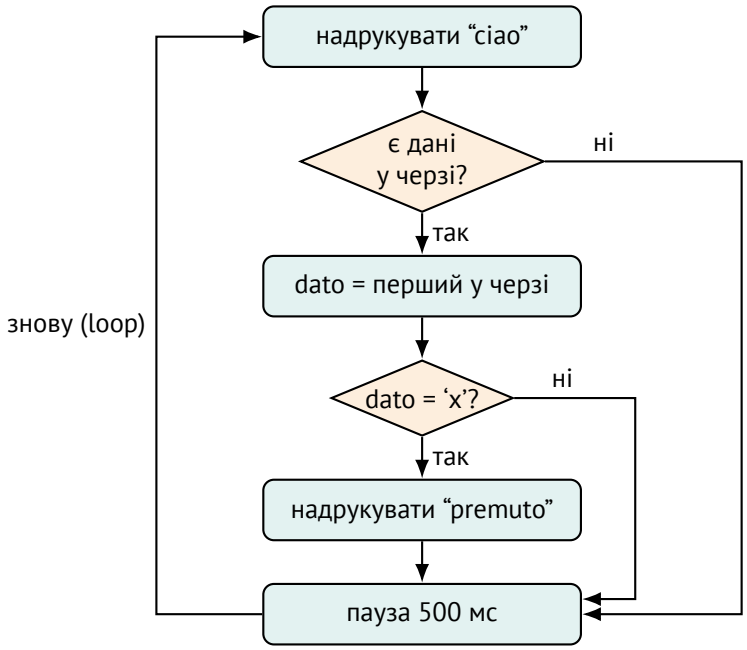


Рис. 1: Блок-схема програми “ciao / premuto”: на кожному колі `loop()` вона друкує “ciao” і лише тоді, коли в черзі є `x`, відповідає “premuto”.

5 Повторення мови С: типи даних і конструкції

Визначення

Біт і **байт**: **байт** складається з **8** **бітів**.
Типи **даних** на Arduino Uno:

Тип	Байти	Значення
`char`	1	від -128 до 127 (символ або мале число)
`byte` (`unsigned char`)	1	від 0 до 255
`int`	2	від $-32\,768$ до $32\,767$
`float`	4	числа **з** **комою**, приблизно 6–7 значущих цифр

Увага

Скільки байтів, а не скільки значень. На питання “зі скількох байтів складається `char`?” відповідь — **1** **байт**, тобто 8 бітів, а не “255”: це стосується того, *які значення* в ньому поміщаються. І увага до діапазону: на Arduino (і на ПК) `char` **зі** **знаком**, тож він від -128 до 127; діапазон від 0 до 255 має `byte` (`unsigned char`). Розмір `int` **залежить** **від** **машини**: на Arduino Uno це 2 байти, на комп’ютері перед вами зазвичай 4.

Визначення

`if` дозволяє **прийняти** **рішення**: якщо відбувається це, то зроби те (“якщо йде дощ, беру парасольку”).
Конструкцій **повторення** (циклів) дві: **`for`** і **`while`**. Усе, що робиться однією, можна зробити й іншою: вибір — це питання **стилю**. Зазвичай програмісти використовують:

- **`for`**, коли **заздалегідь** знають, скільки разів повторювати (“увімкни світлодіод 10 разів”);
- **`while`**, коли кількість повторень **не відома наперед** і залежить від зовнішніх подій, тобто з’ясовується лише *під час* виконання (*runtime*).

Увага

Ці теми треба **повторити зараз**: на наступних уроках `for`, змінні й типи даних використовуватимуться одразу, без пояснення щоразу. Вони будуть і в **контрольній з повторення** 29 жовтня.

6 Функції

Приклад

Спецприйом у відеоіграх. У старих файтингах, щоб зробити спецприйом, треба було швидко натиснути послідовність клавіш — наприклад, вниз, вниз, вліво, вправо, удар рукою, удар ногою — і потрібна була певна спритність рук. Сьогодні багато контролерів дозволяють **призначити всю послідовність одній кнопці**: натискаєш її — і прийом виконується повністю.
Функція — саме це: **набір операцій**, якому дають **назву**. Щоразу, коли в програмі пишуть цю назву, машина виконує всі операції набору, і їх не треба переписувати.

Приклад

П’ять євро і бутерброд. Уяви помічника, якого просять лише про одне: даєш йому 5 євро, і через деякий час він повертається з бутербродом. Як він це робить (іде до автомата або на кухню, ріже хліб, кладе ковбасу й сир, додає соус) тебе **не цікавить**: хтось пояснив йому це раз і назавжди. Тебе цікавить лише, що ти **даєш 5 євро** і **отримуєш бутерброд**.
Функцію в C пишуть так само:

```
Panino comprapanino(int soldi) {   // схема для розуміння, не код для компіляції
  // іде на кухню, ріже хліб, ... готує бутерброд
}
```

- ліворуч — **що повертає** функція (**тип** результату: тут бутерброд, *panino*);
- потім **назва**, яку ми даємо функції;
- у круглих дужках — **що ми їй даємо** (**параметри**: тут гроші, *soldi*);
- у фігурних дужках — **як вона це робить**.

Визначення

`setup` і `loop` теж функції: **`void setup()`**. **`void`** означає “порожньо”: функція **нічого не повертає**, вона лише має щось виконати. А порожні дужки кажуть, що ми не даємо їй **жодного параметра**. Те саме для `loop`.

6.1 Функція, яка блимає світлодіодом

Одне блимання (*lampeggio*) робиться так: `digitalWrite` високий, `delay`, `digitalWrite` низький, `delay`. Це можна загорнути у функцію: вона нічого не повертає, тож вона `void`.

```
void lampeggio() {
  digitalWrite(13, HIGH);
  delay(500);
  digitalWrite(13, LOW);
  delay(500);
}
```

У `loop` досить написати **`lampeggio();`**: коли програма доходить до цього місця, вона **перескакує** всередину функції, виконує чотири команди, а потім **продовжує** звідти, де зупинилася. Це як вставити в код маленький шматочок, який можна викликати, коли потрібно.

6.2 Додаємо параметри: який вивід, скільки разів, як швидко

Так функція блимає *завжди* виводом 13. Її можна **налаштувати**, давши їй інформацію, тобто **параметри**:

```
void lampeggio(int pin) {
  digitalWrite(pin, HIGH);
  delay(500);
  digitalWrite(pin, LOW);
  delay(500);
}
```

Якщо написати **`lampeggio(13);`**, число 13 **копіюється** в `pin`, і блимає вивід 13; якщо одразу нижче написати **`lampeggio(7);`**, блимає вивід 7. Код функції не змінився: змінився **параметр**, вибраний ззовні. Назву `pin` ми вибрали самі: вона могла бути будь-якою.

Ще один крок: також **скільки** **разів** блимати. Тут потрібен `for` (ось чому його треба повторити):

```
void lampeggio(int pin, int n) {
  for (int i = 0; i < n; i++) {
    digitalWrite(pin, HIGH);
    delay(500);
    digitalWrite(pin, LOW);
    delay(500);
  }
}
```

Примітка

У `for`: `int i = 0` — **початкова** **умова**; `i < n` — **умова** **повторення** (повторюємо, поки `i` менше за `n`); `i++` **збільшує** `i`. Отже, `i` набуває значень 0, 1, 2, … до $n-1$: блок виконується рівно `n` разів.

З двома параметрами, розділеними **комою**, у `loop` пишуть `lampeggio(13, 3);` і `lampeggio(7, 9);`: перше число потрапляє в `pin`, друге — в `n`. Вивід 13 блимає 3 рази, вивід 7 — 9 разів.

Приклад

Скільки коду заощаджуємо. Без функцій, щоб вивід 13 блимнув 3 рази, а вивід 7 — 9 разів, треба було написати чотири команди блимання у двох різних `for`, приблизно 10 рядків; з функцією досить її рядків і двох викликів. І **з ростом вимог код не росте**: щоб вивід 5 блимнув 100 разів, досить додати `lampeggio(5, 100);`.

Примітка

Функцію пишуть **поза** `setup()` і `loop()`: до або після, але ніколи всередині. У середовищі Arduino (і в Tinkercad) вона може стояти й після `loop()`, як у коді уроку.

6.3 Функції, які повертають результат

`lampeggio` нічого не повертає. А функція може зробити **обчислення** і **повернути результат**: тоді ліворуч від назви замість `void` пишуть **тип даних**, що повертається (`char`, `int`, `float`, ...).

Приклад

Відстань між двома точками. На координатній площині є дві точки, (x_1, y_1) і (x_2, y_2). Якщо провести проєкції на осі, утвориться прямокутний трикутник: два **катети** дорівнюють $x_2 - x_1$ і $y_2 - y_1$, а відстань між точками — це **гіпотенуза**. За **теоремою Піфагора**:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Функція, яка робить це обчислення (*calcolo*):

```
float calcolo(int x1, int y1, int x2, int y2) {
  int dx = x2 - x1;              // delta x: різниця
  int dy = y2 - y1;              // delta y
  int ipotenusa2 = dx*dx + dy*dy; // квадрат гіпотенузи
  float r = sqrt(ipotenusa2);    // квадратний корінь
  return r;                      // повертає результат
}
```

Команду **`return`** ставлять у кінці функції, і вона каже, **яке значення повернути**. У C квадратний корінь пишеться **`sqrt`**.
У `loop`: `float distanza = calcolo(2, 1, 6, 4);`. Перша точка $(2, 1)$, друга $(6, 4)$. Машина обчислює $dx = 6 - 2 = 4$, $dy = 4 - 1 = 3$, потім $4 \cdot 4 + 3 \cdot 3 = 16 + 9 = 25$, і нарешті $\sqrt{25} = 5$: у змінну `distanza` потрапляє **5**.

Примітка

Перевага функцій: **довгу і складну** процедуру пишуть **лише один раз**, і тому, хто її використовує, не треба пам'ятати, як вона робиться. Функції на зразок `sqrt`, синуса й косинуса написав хтось інший, можливо, десятиліття тому, і відтоді ми користуємося ними, не переписуючи: хто програмує калькулятор, не переписує обчислення синуса, а використовує готову функцію.

Примітка

Є також функції, які **повертають значення** без **жодного параметра**: наприклад, `millis()`, яка каже, скільки мілісекунд минуло від увімкнення. На уроці наводили приклад функції, що генерує **випадкове число**: *уточнення*: в Arduino вона називається `random` і потребує щонайменше одного параметра, межі (`random(10)` дає число від 0 до 9).

7 Мітки: `#define`

У схемі уроку червоний світлодіод на виводі 3, а зелений на 5. Якщо писати `pinMode(3, OUTPUT)`, `digitalWrite(5, HIGH)`, …, щоразу треба **пам'ятати**, хто 3, а хто 5. Можна дати числу **назву**:

```
#define RED_LED 3
#define GREEN_LED 5
```

Відтоді можна писати `pinMode(RED_LED, OUTPUT)` замість `pinMode(3, OUTPUT)`.

Визначення

`#define` створює **мітку** (*label*, *etichetta*): це **не змінна**, а назва, дана числу, яку перед компіляцією замінюють самим числом. Не ставлять ні знак =, ні крапку з комою в кінці.

Приклад

Навіщо це. У кількох рядках коду користі не видно. Але в програмі на 10 000 рядків, якщо в рядку 7422 бачиш `digitalWrite(14, HIGH)`, то 14 — це який світлодіод? Зелений, синій, білий? Треба повертатися на початок або дивитися на схему. А якщо бачиш `digitalWrite(LED_BIANCO, HIGH)`, усе зрозуміло одразу.
Це як **телефонна книга**: щоб комусь подзвонити, ти шукаєш не номер, а **ім'я**. Так само для довгих і важких для запам'ятовування чисел, як число пі: йому дають назву і використовують її замість числа. Для числа пі 3,14 — це мало точно: щоб розрахувати траєкторію снаряда до Місяця, потрібно чимало цифр, до Марса — ще більше, бо чим далі, тим точнішим має бути розрахунок. З міткою довге число пишуть лише один раз.

Увага

- У назві використовують **нижнє підкреслення** (`RED_LED`), **а не мінус**: `RED-LED` читалося б як віднімання `LED` від `RED`.
- За домовленістю мітки пишуть **великими літерами**. Як в електронних листах і повідомленнях, де великі літери означають "кричати", так і в C великі літери мають точне значення. У C великі літери означають **сталу** величину, яка ніколи не змінюється; малі — те, що може змінюватися під час виконання (як змінні). Працює й малими, але гарний стиль вимагає великих.

8 Практика в Tinkercad: два світлодіоди, три параметри

Безпека праці

Схему збирають у **симуляції** (Tinkercad), але правила ті самі, що й для справжньої схеми: **провід**, що з'єднує **вивід** **output** прямо **з** **5 В**, створює **коротке** **замикання**, коли вивід переходить на низький рівень, і на справжній платі може **пошкодити** **Arduino**. Завжди дотримуйтеся схеми.

Схема. У Tinkercad беруть **Arduino** **Uno** і **макетну** **плату** (*breadboard*), як завжди під'єднують лінії живлення (5 В і GND) і ставлять два світлодіоди: **червоний** **на** **вивід** **3**, **зелений** **на** **вивід** **5**. Кожен світлодіод має свій **резистор** **на** **220 Ω**: вивід іде до резистора, резистор — до **анода** світлодіода, **катод** — до GND.

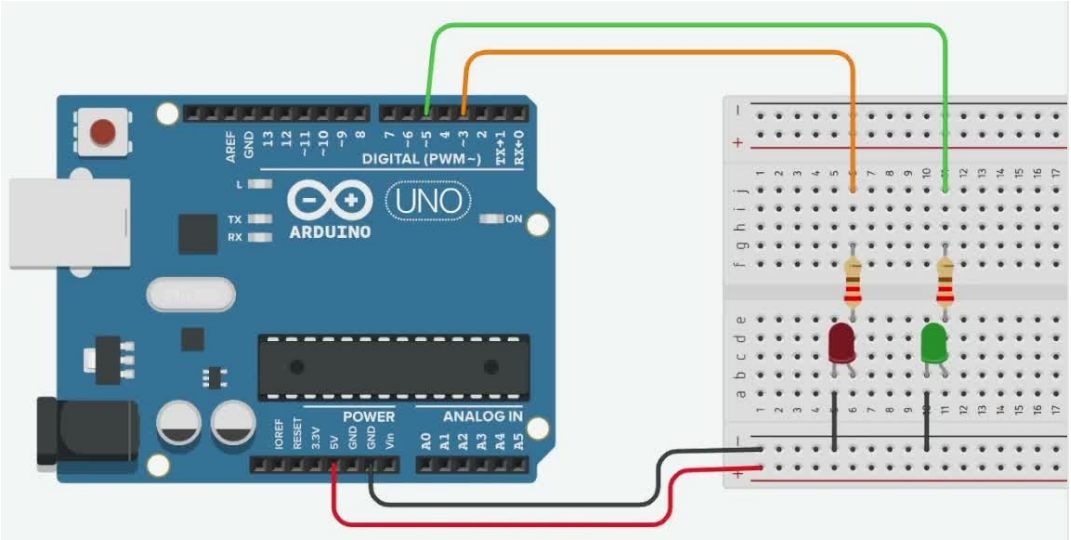


Рис. 2: Схема в Tinkercad: помаранчевий провід веде від виводу 3 до червоного світлодіода, зелений провід — від виводу 5 до зеленого світлодіода; кожен світлодіод має свій резистор, а катоди під'єднані до лінії GND макетної плати.

Увага

Помилки, які траплялися під час практики:

- **світлодіод,** **поставлений** **навпаки** (анод і катод переплутано), **не** **світиться**: анод має бути до резистора й виводу, катод — до GND;
- **зайві** **проводи** між резистором і 5 В, яких немає на схемі: вони не потрібні і створюють описане вище коротке замикання. Завжди порівнюйте своє креслення зі схемою.

Код. Спочатку з двома параметрами (`lampeggio(RED_LED, 7)` і `lampeggio(GREEN_LED, 3)`): червоний світлодіод блимає 7 разів, потім зелений 3 рази, потім усе спочатку. Потім завдання: додати **третій** **параметр**, **час** `T`, щоб червоний блимав 7 разів швидко (200 мс), а зелений 3 рази повільно (700 мс). Розв'язок (код однаковий для всіх мов):

```
/*
    sul pin 3 abbiamo il led rosso
    sul pin 5 abbiamo il led verde
*/
#define RED_LED 3
#define GREEN_LED 5

void setup()
{
    pinMode(RED_LED, OUTPUT);
    pinMode(GREEN_LED, OUTPUT);
}

void loop()
{
    lampeggio(RED_LED, 7, 200);
    lampeggio(GREEN_LED, 3, 700);
}

void lampeggio(int pin, int N, int T)
{
    for (int i = 0; i < N; i++)
    {
        digitalWrite(pin, HIGH);
        delay(T);
        digitalWrite(pin, LOW);
        delay(T);
    }
}
```

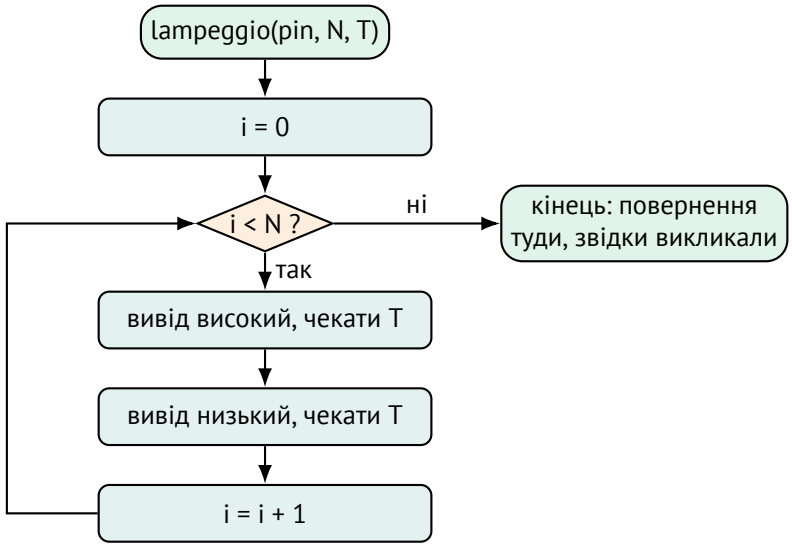


Рис. 3: Блок-схема функції `lampeggio(pin, N, T)`: блок увімкнути/вимкнути повторюється N разів, кожна фаза триває T мілісекунд.

Під час симуляції червоний світлодіод швидко блимає 7 разів, а зелений повільно 3 рази. Порівняно з попередньою версією додали лише **кілька маленьких шматочків**: структура коду залишилася тією самою.

Примітка

Чому два світлодіоди не блимають разом? Бо програма виконує команди **одну за одною**: поки функція блимає червоним, вона не може займатися зеленим. Щоб дві речі працювали "одночасно", потрібен зовсім інший спосіб писати код: ми розглянемо це наступного року.

Примітка

У симуляції видно, що світлодіод гасне не миттєво: і в реальності світлодіод має невеликий **час вимикання**.

9 Що запам'ятати з цього уроку

- Програма Arduino має **`setup()`** (виконується один раз) і **`loop()`** (виконується безперервно).
- **`pinMode`** задає напрямок виводу; **`digitalWrite`** записує на вихід; **`digitalRead`** читає вхід. **`delay`** зупиняє машину на час у мілісекундах (лише цілі числа).
- **`Serial`**: `begin` (однакова швидкість з обох боків), `print`/`println`, `available` (скільки в черзі в буфері), `read` (читає і прибирає з черги).
- Типи даних: `char` 1 байт ($-128 \ldots 127$), `int` 2 байти на Arduino Uno, `float` 4 байти, з комою. `for`, якщо кількість повторень відома наперед, `while`, якщо вона залежить від зовнішніх подій.
- **Функція** — набір операцій з назвою: тип, що повертається (або `void`), назва, **параметри** в дужках і **`return`**, щоб повернути результат.
- Завдяки параметрам та сама функція працює в багатьох випадках: `lampeggio(pin, N, T)` блимає будь-яким виводом, скільки завгодно разів і з потрібною швидкістю.
- **`#define`** дає числу назву: пишуть великими літерами, з нижнім підкресленням, без `=` і без крапки з комою.

Приклад

До контрольної з повторення 29 жовтня повторіть усе, що вище: основні команди Arduino, `Serial`, змінні й типи даних, `if`, `for` і `while`, функції з параметрами і з поверненням значення. На наступній контрольній з електроніки буде також питання, **чому** резистор світлодіода має **220 Ω**: повторіть це.

Глосарій: італійські слова, які вживають на уроці

Italiano	**Українською**
piedino	вивід (pin)
ingresso / uscita	вхід (input) / вихід (output)
acceso / spento	увімкнено / вимкнено
ritardo, pausa (delay)	затримка, пауза
coda, buffer	черга, буфер
velocità	швидкість
tipo di dato	тип даних
ripetizione, ciclo	повторення, цикл
funzione, parametro	функція, параметр
riconsegnare (return)	повертати (return)
lampeggio, lampeggiare	блимання, блимати
etichetta	мітка
resistore, LED	резистор, світлодіод
anodo / catodo	анод / катод
cortocircuito	коротке замикання
verifica di ripasso	контрольна з повторення
griglia	сітка
fori di fissaggio	кріпильні отвори